

Understanding Unified Modelling Language (UML)

UML is a modeling language

- A **modeling language** allows the specification, the visualization, and the documentation of the development of a software system
- The **models** are *artifacts* which clients and developers use to communicate
- UML 1.* is a modeling language
- UML 2.* is also a programming language

Meaning of models for software

- A model is a *description* of the structure and meaning of a system
- A model is always an *abstraction* at some level: it captures the essential aspects of a system and ignores some details
- **Important:** a model can be also *generator* of potential *configurations* of systems

Roots of UML

At the beginning of the '90 there was a convergence:

- **Booch method** (Grady Booch)
 - **OMT** (Jim Rumbaugh)
 - **Fusion/OOSE** (Ivar Jacobson)
- } '94 – join *Rational Software Corporation*
- '95 – joins Rational

Evolution of UML

- OO languages appear, since mid 70' s to late 80' s
- Between ' 89 and ' 94, OO methods increased from 10 to 50
- Unification of ideas began in mid 90' s
 - 1994 Rumbaugh joins Booch at Rational
 - 1995 v0.8 draft Unified Method
 - 1995 Jacobson joins Rational (*Three Amigos*)
 - 1996 June: UML v0.9 published
 - 1997 Jan: UML 1.0 offered to OMG
 - 1997 Jul: UML 1.1 OMG standard
 - 1998: UML 1.2
 - 1999: UML 1.3
 - 2001: UML 1.4
 - 2003 Feb: IBM buys Rational
 - 2003: UML 1.5
 - 2004: UML 1.4.2 becomes the standard ISO/IEC 19501
 - 2005: UML 2.0
 - 2007: UML 2.1.2
 - 2009: UML 2.2
 - 2010: UML 2.3
 - 2011: UML 2.4
 - 2013: UML 2.5

pre-UML

UML 1.x

UML 2.0

OMG

- Object Management Group, founded in 1989
- Consortium of 800 industries (eg. IBM, HP, Apple, etc.) and many universities.
- Produces specifications of reference architectures, eg. CORBA
- Other specifications: UML, various MDA technologies
- UML as managed by OMG is a standard de facto in continuous evolution.
- UML1.4 (onwards) is a ISO standard.

Main UML specification documents

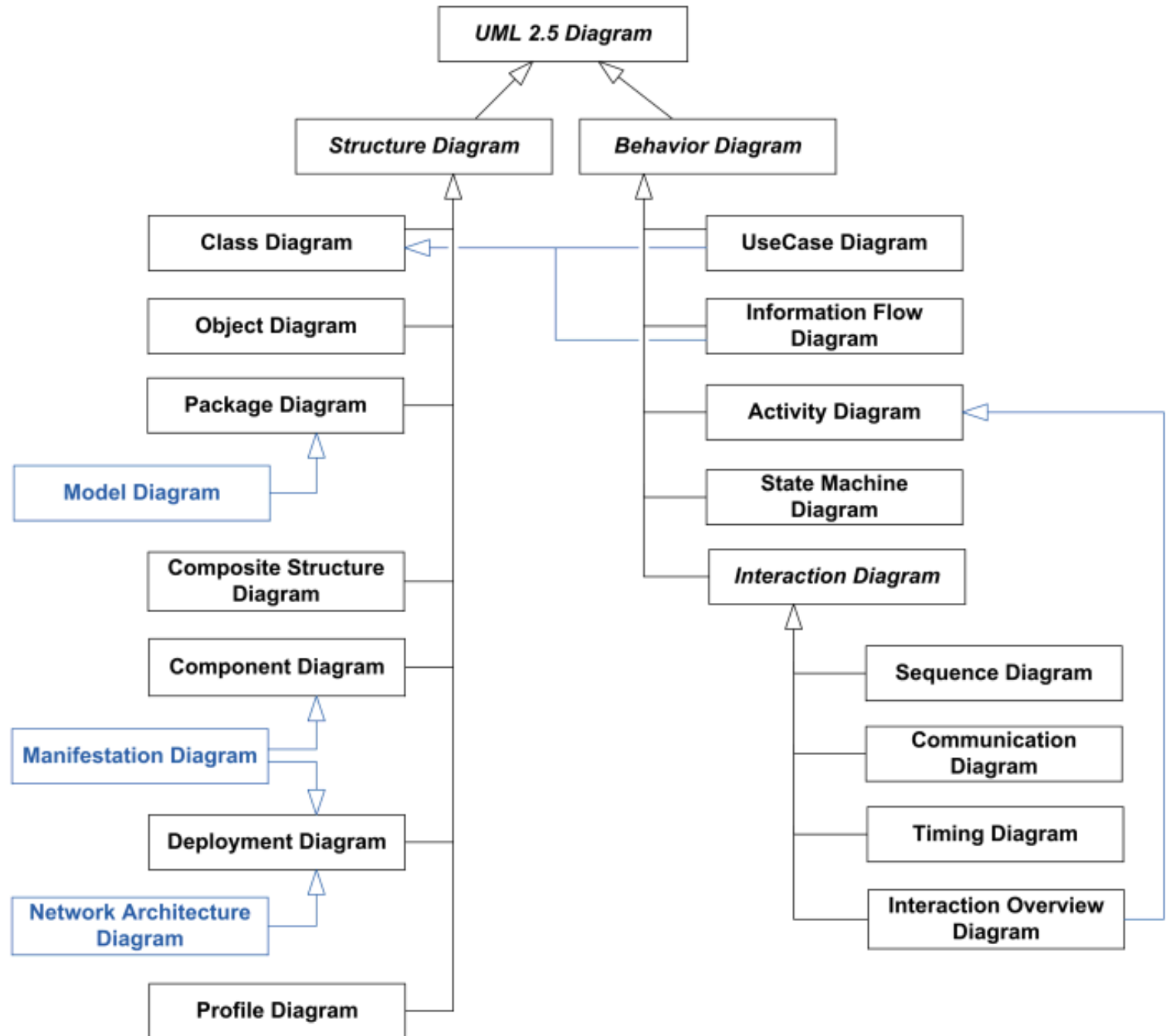
- **Superstructure**: defines the UML elements (diagrams, etc.)
- **Infrastructure**: defines the UML core metamodel
- **OCL** (Object Constraint Language):
formal language for writing predicates, constraints, and formulas inside diagrams
- **XMI** (XML Metadata Interchange):
Document Type Definition (DTD) for UML models
- UML Diagram Interchange: XMI + graphic info

Canonical diagrams (Superstructure, vers. 2.4)

Version 2.4 includes 14 canonical diagrams

- **Structure**
 1. Class
 2. Composite structure
 3. Component
 4. Deployment
 5. Object
 6. Package
 7. Profiles (added in version 2.4)
- **Behavior**
 1. Activity
 2. Statecharts
 3. Usecase
- **Interaction**
 1. Communication
 2. Interaction Overview
 3. Sequence
 4. Timing

UML 2.5 (2013)



Structure and Behavior

- UML focesses on two aspects of object oriented models:
 - Structure and Behavior
- It aims at visualizing both.

Describing the structure of software

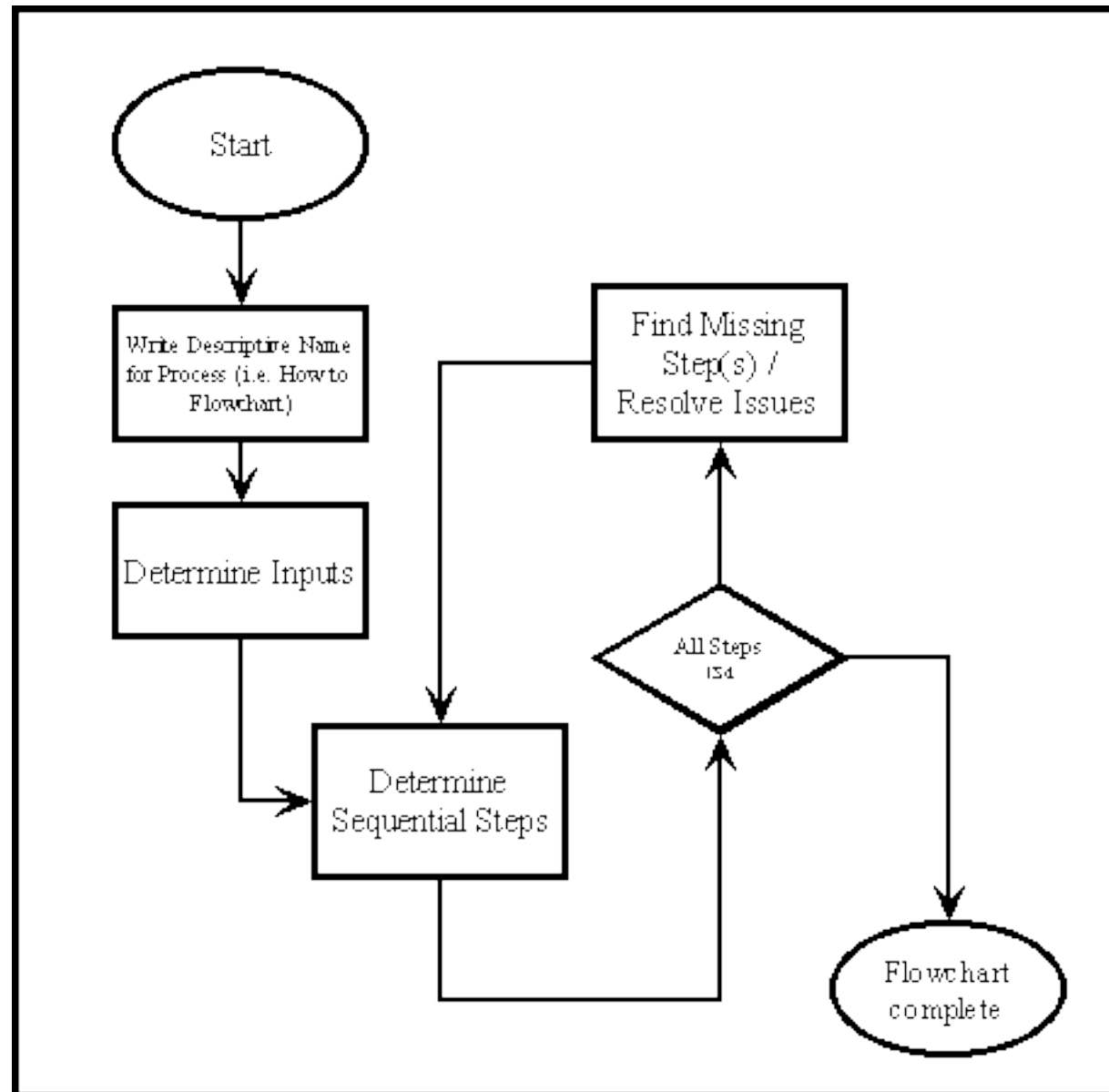
- The description of structure offers an account of what a system is made of, in terms of both its parts and the relationships among them.
- A structure may be a hierarchy featuring one-to-many relationships, or
- A network featuring many-to-many links.
- In short, artifacts and the way they are connected to each other to define a system

Discuss

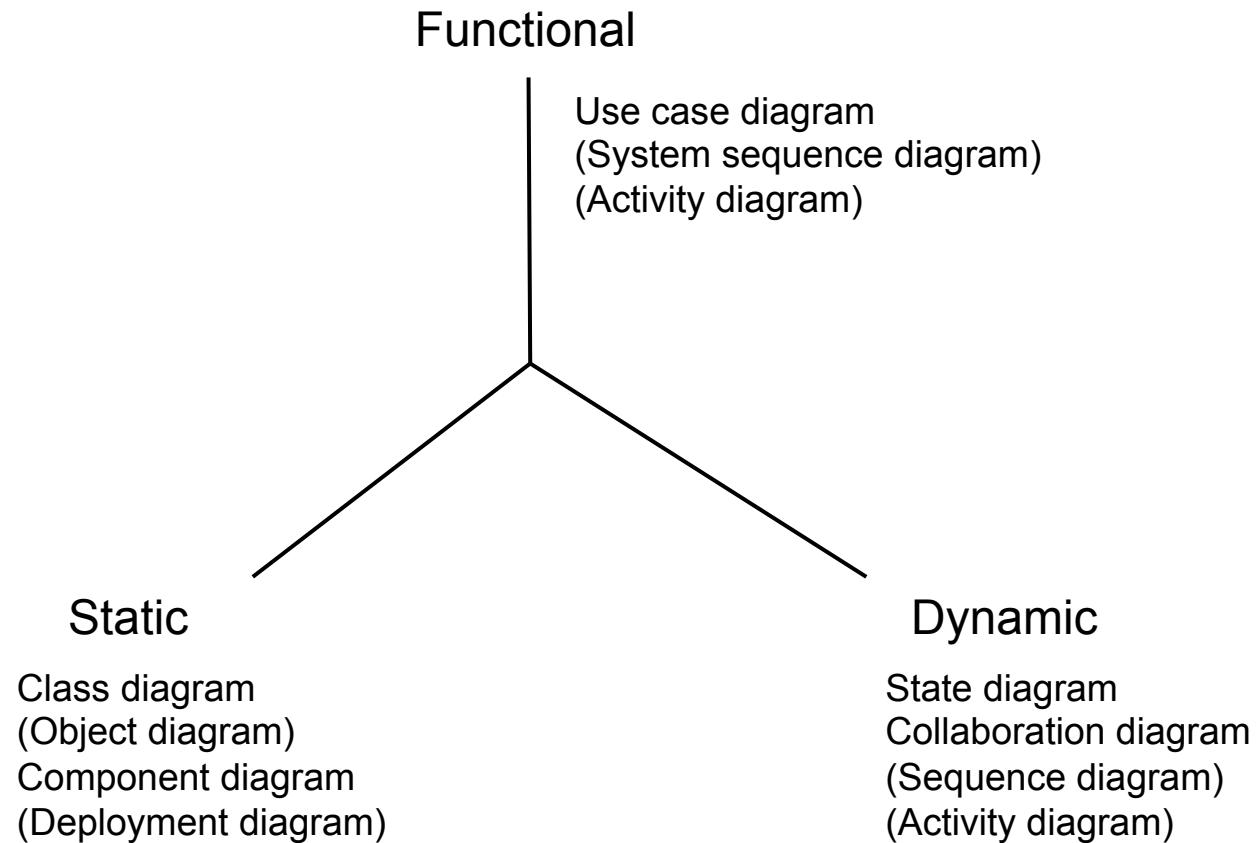
- Which ways do you know to pictorially describe “behaviors” - or actions?



Flowchart

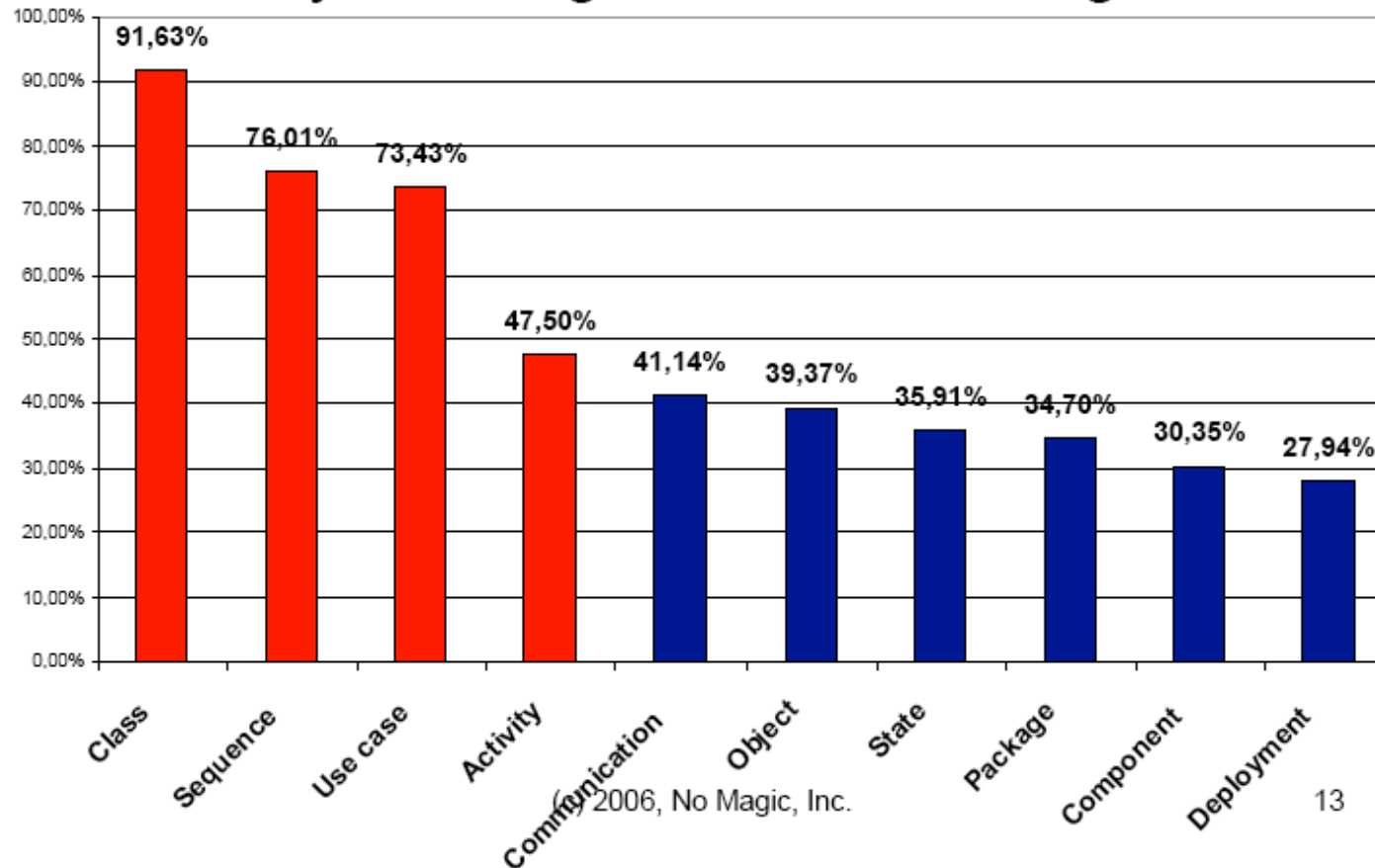


Three modeling axes



Usage survey

Survey on Usage of UML 1.x Diagrams



Main diagrams

The main diagrams that are used in most views are :

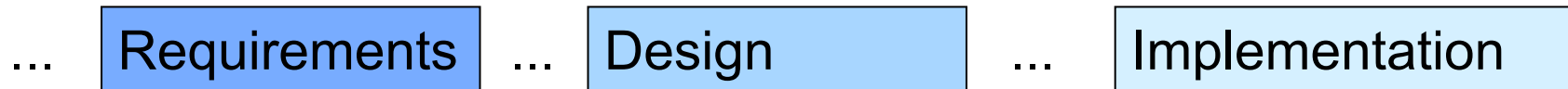
- **Use case** diagram
- **Class** diagram
- **Sequence** diagram
- **Activity** diagram

Discuss

- Which diagrams are most useful in each lifecycle phase?



Diagrams in lifecycle



Use Case

Class diagram

Sequence diagram

Activity diagrams and Statecharts

Diagrams during design

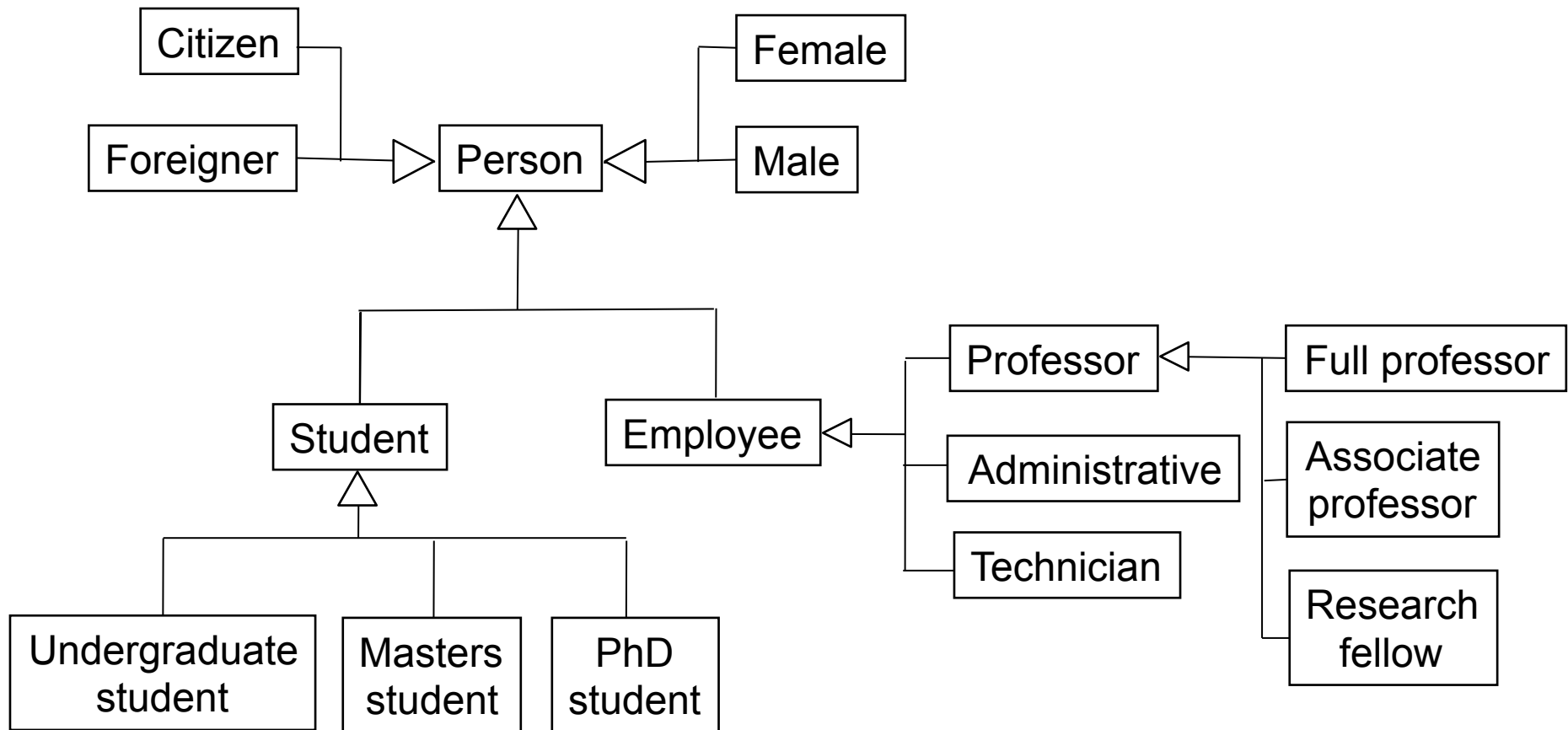
Three main types:

- Class diagrams for domain entities and data structures
- Sequence diagrams for multiple objects interactions via messages
- Statecharts for behaviors and algorithms of a single object

Example

- A university is an organization where some persons work, some other study
- There are several types of roles and grouping entities
- We say nothing about behaviors, for the moment

A taxonomy

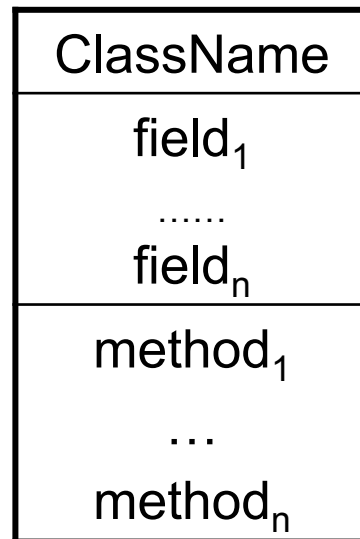


Class diagram

- Most common diagram in OO modeling
- A class diagram is a graph including:
 - Nodes representing **classes (types of objects)**
 - Nodes can have just a name, or expose some internal structure
 - Links representing **relationships among classes**
 - Inheritance
 - Association
 - Aggregation or composition
 - Dependency
 - Links can have multiplicities and/or names for roles played by participants

Notation for classes

- The notation for classes is a rectangular box with three compartments



The top compartment shows the class **name**

The middle compartment contains the declarations of the fields, or *attributes*, of the class

The bottom compartment contains the declarations of the *methods* of the class

Example

A point defined by classes at three different abstraction levels

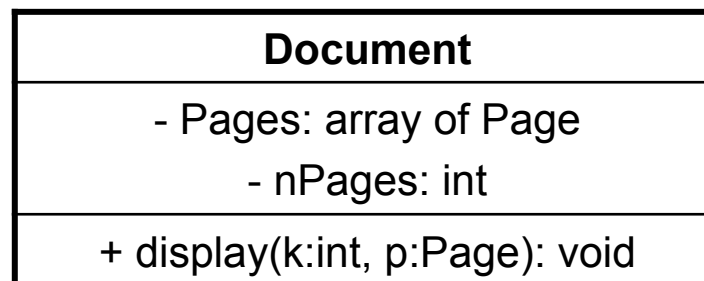
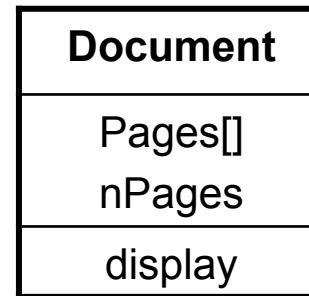
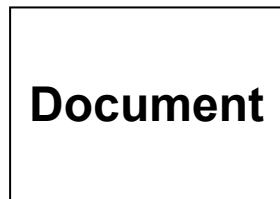
Point

Point
x
y
Move

Point
- x: int - y: int
+ move(dx: int, dy: int): void

Example

A document defined by classes at three different abstraction levels



UML Class Diagrams

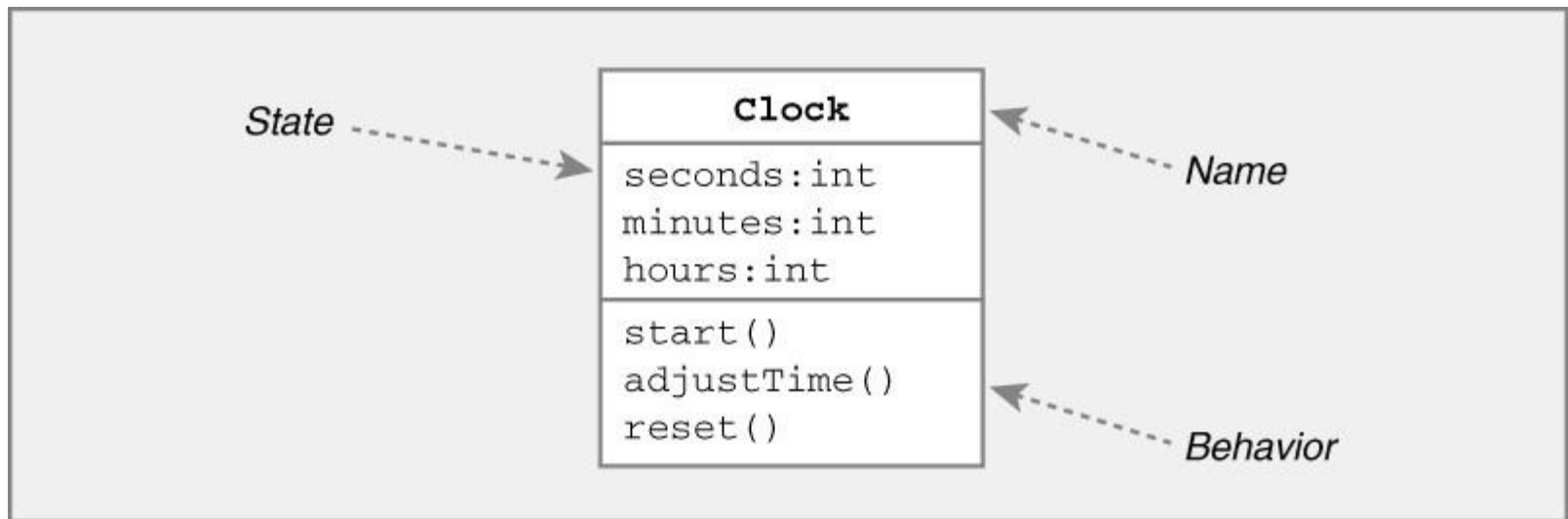
■ Represent the (static) structure of the system

■ General In Java

■ Name Name

■ State Variables

■ Behavior Methods



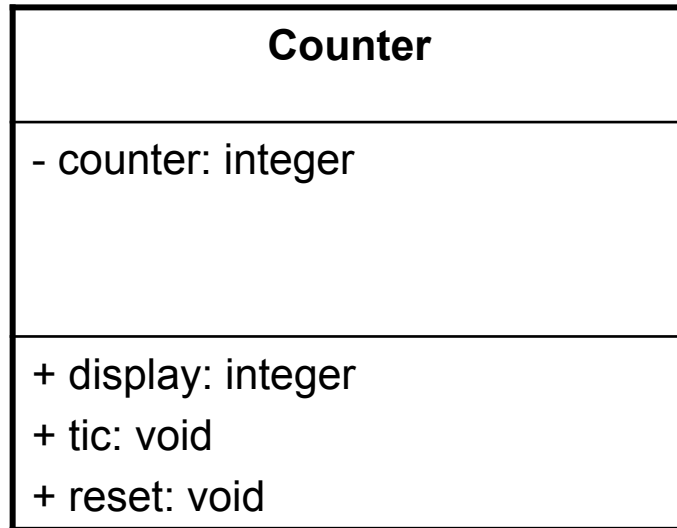
Exercise



Draw a class diagram for the following Java code

```
class Person {  
    private String name;  
    private Date birthday;  
    public String getName() {  
        // ...  
    }  
    public Date getBirthday() {  
        // ...  
    }  
}
```

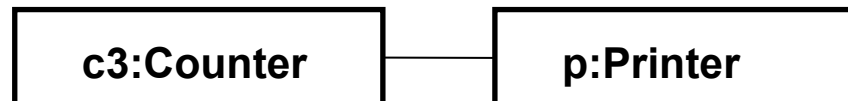
A counter class



A class in UML

```
class Counter{  
    private counter: integer;  
    public integer display()  
        {return counter};  
    public void tic()  
        {counter = counter + 1};  
    public void reset()  
        {counter = 0};  
}
```

A corresponding class
in a programming language



Using an object of type class
in an object oriented system

Relationships Between Classes

■ **Association**  OR 

- Permanent, structural, “has a”
- Solid line (arrowhead optional)

■ **Dependency** 

- Temporary, “uses a”
- Dotted line with arrowhead

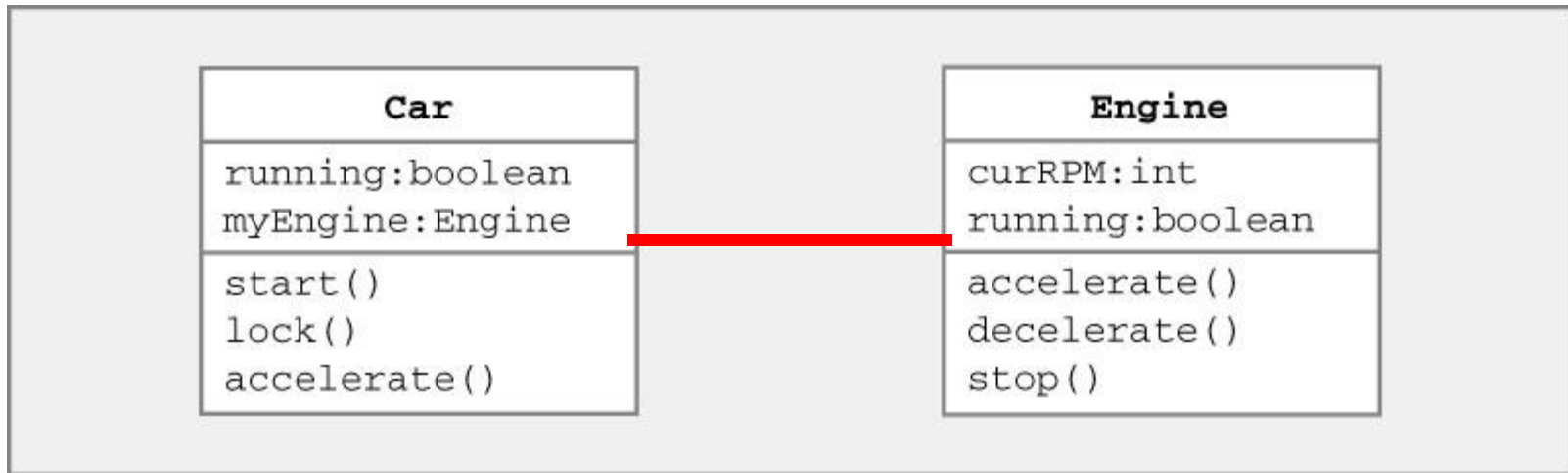
■ **Generalization** 

- Inheritance, “is a”
- Solid line with open (triangular) arrowhead

■ **Composition and Aggregation**

Association

- Denotes permanent, structural relationship
- State of class A contains class B
- Represented by solid line (arrowhead optional)



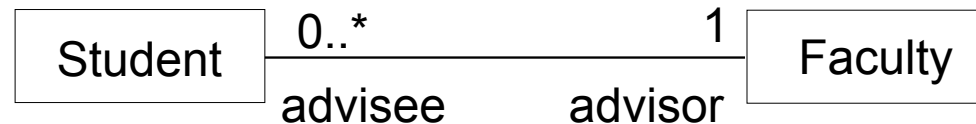
Car and Engine classes know about each other

Association and roles

- The simplest relationship among classes is the *association*
- An association represents a structural relationship that connects two classes
- It means that the two classes “know” each other: all objects in a class have some relationship with some object(s) in the other class
- An association can have a name, which usually describes its role

Roles and multiplicity

- An association line may have a *role name* and a *multiplicity specification*
- The multiplicity specifies an integer interval, e.g.,
 - $l..u$ closed (inclusive) range of integers
 - i singleton range
 - $0..*$ nonnegative integer, i.e., 0, 1, 2, ...



Multiplicity of Associations

- Some relationships may be quantified
- Multiplicity denotes how many objects the source object can legitimately reference
- Notation
 - * $\Rightarrow$ 0, 1, or more
 - 5 $\Rightarrow$ 5 exactly
 - 5..8 $\Rightarrow$ between 5 and 8, inclusive
 - 5..* $\Rightarrow$ 5 or more

Multiplicity of Associations

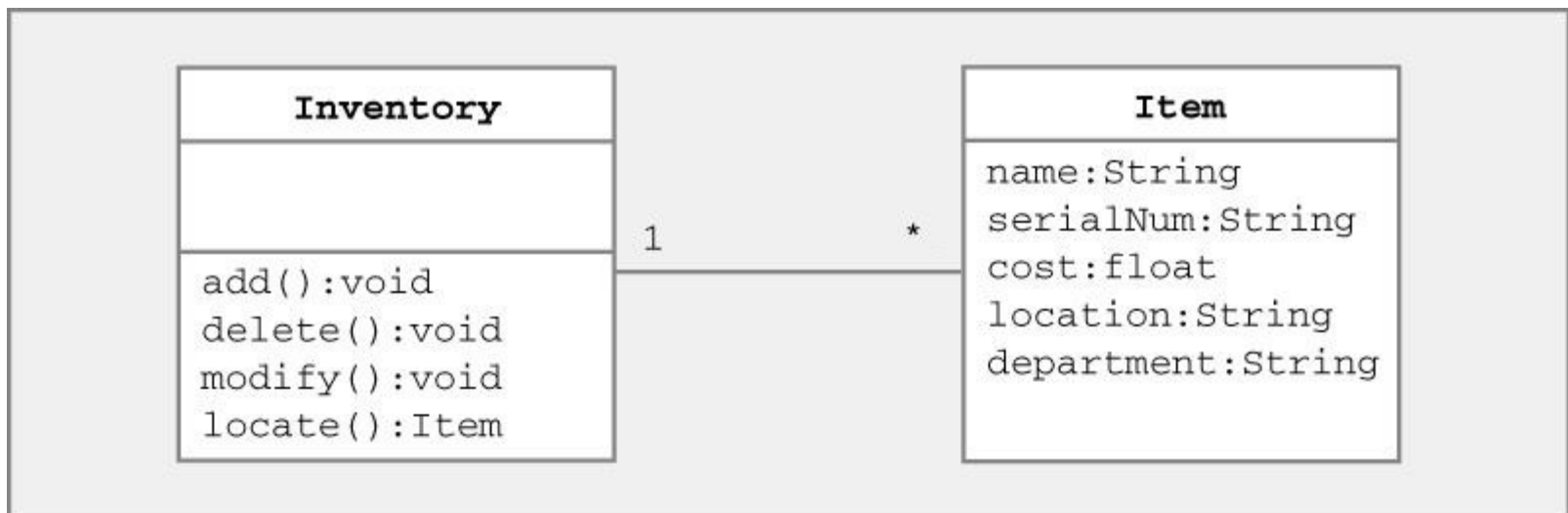
■ Many-to-one

- Bank has many ATMs, ATM knows only 1 bank



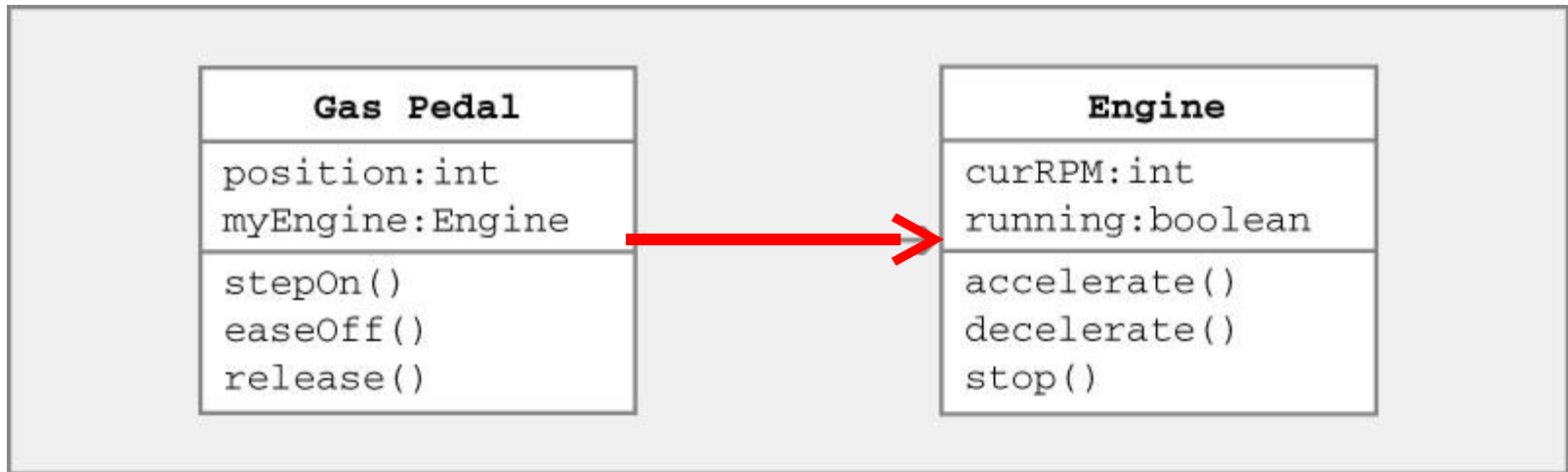
■ One-to-many

- Inventory has many items, items know 1 inventory



Associations w/ Navigation Information

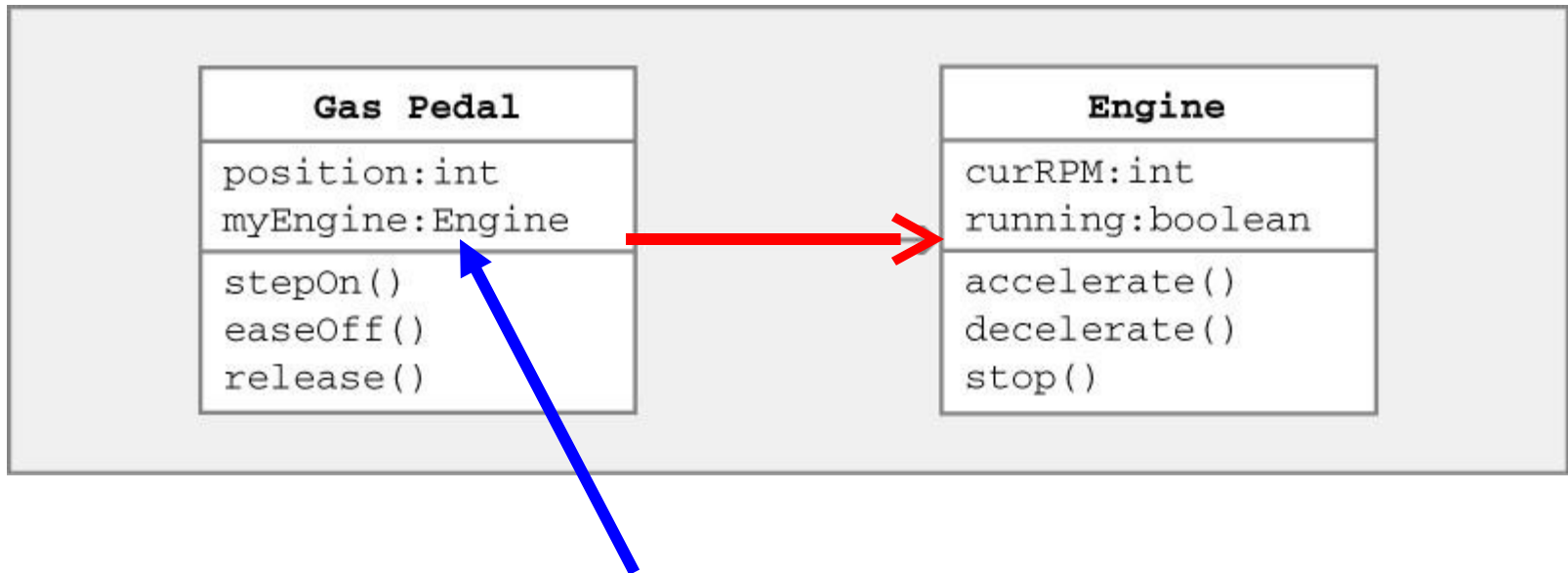
- Can indicate direction of relationship
- Represented by solid line with arrowhead



Gas Pedal class knows about Engine class
Engine class doesn't know about Gas Pedal class

Associations w/ Navigation Information

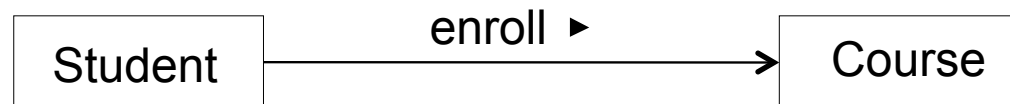
- Denotes “**has-a**” relationship between classes
- “Gas Pedal” **has a** “Engine”



State of Gas Pedal class contains instance of Engine class ⇒ can invoke its methods

Association with direction

- An association may have an optional label consisting of a name and a direction arrow
- The direction arrow indicates the direction of association with respect to the name

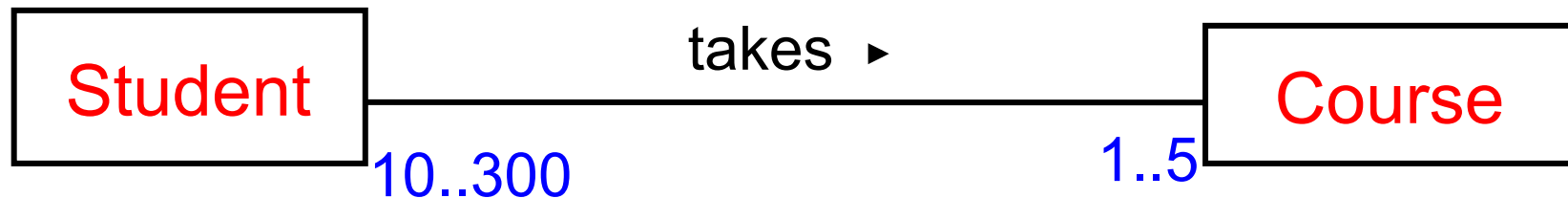


An arrow may be attached to the end of path to indicate that *navigation* is supported in that direction:

Student knows Course, Course does not know Student

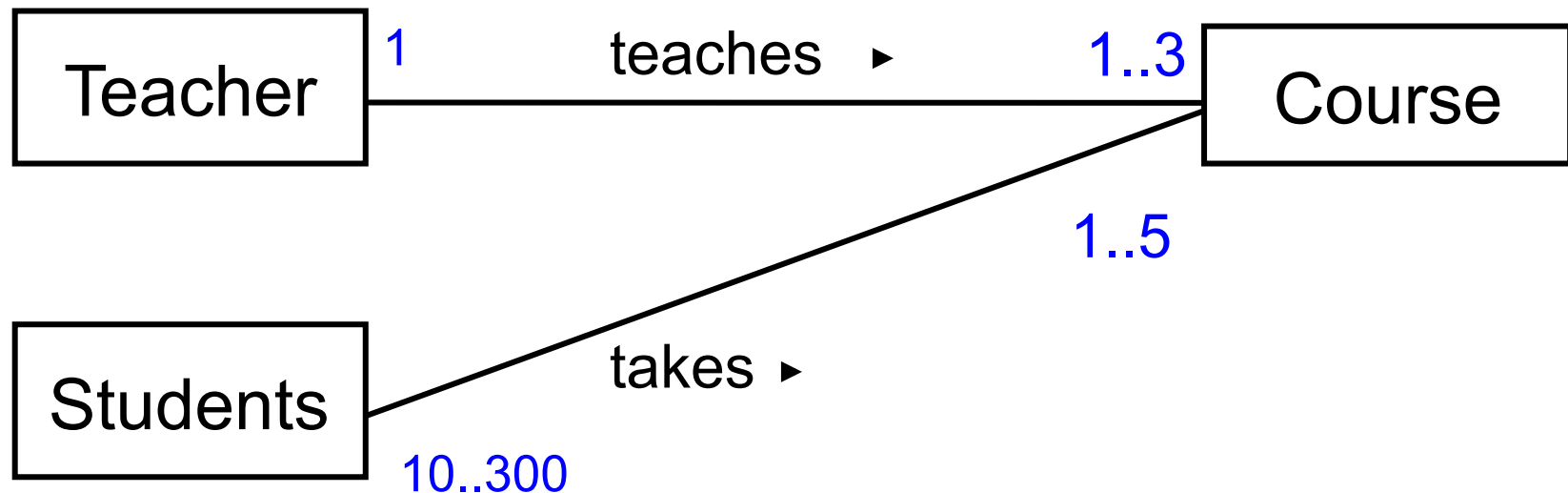
Association example

- A **Student** can take up to **five Courses**
- Every Student has to be enrolled in at least **one** course
- Up to **300** students can enroll in a course
- A class should have **at least 10** students



Example

- A teacher teaches 1 to 3 courses
- Each course is taught by only one teacher
- A student can take between 1 to 5 courses
- A course can have 10 to 300 students



Exercise



Explain the meaning of this diagram

